

Diffusion Models for Object Detection: Complete Technical Survey

Based on Source Materials

September 7, 2026

Abstract

This report provides a complete technical survey of diffusion-based object detection methods, covering both DiffusionDet (2D object detection) and Diff3Det (3D object detection). All content, definitions, formulas, derivations, examples, and comparisons are taken directly from the source materials without addition. The survey covers: (1) the evolution of object detection from empirical object priors to learnable queries to diffusion-based methods; (2) DiffusionDet’s random box approach with dynamic number of boxes and iterative evaluation; (3) the diffusion model formulation including forward and reverse processes; (4) the complete DiffusionDet architecture with image encoder and detection decoder; (5) DDIM sampling and box renewal mechanisms; (6) DETR vs DiffusionDet comparison; (7) Diff3Det for 3D object detection in point clouds; (8) BEV box representation $(c_x, c_y, d_x, d_y, \theta)$; (9) correlation coefficient constraint $W = \rho L + \sqrt{1 - \rho^2} X$; (10) dynamic time step schedule; (11) bipartite matching with $C_{\text{match}} = \lambda_{\text{cls}} L_{\text{cls}} + \lambda_{\text{reg}} L_{\text{reg}} + \lambda_{\text{IoU}} L_{\text{BEV IoU}}$; (12) training loss with $L = \lambda_{\text{cls}} L_{\text{cls}} + \lambda_{\text{reg}} L_{\text{reg}} + \lambda_{\text{IoU}} L_{\text{DIOU}}$; (13) inference sampling equations; (14) experiments on COCO, CrowdHuman, and KITTI datasets; (15) ablation studies; and (16) the core intuition of diffusion-based detection in both 2D and 3D.

Contents

1	Introduction: The Evolution of Object Detection	6
1.1	Traditional Object Detection	6
1.2	Empirical Object Priors	6
1.3	The Evolution of Object Candidates	6
1.4	Sliding Windows	7
1.5	Region Proposals	7
1.6	Anchor Boxes	7
1.7	Reference Points	7
1.8	Learnable Object Queries	7
1.9	The Core Change	8
1.10	Object Detection Definition	8
2	Diffusion Models: Background	8
2.1	What is a Diffusion Model?	8
2.2	The Forward Process	8

2.3	Reparameterization	9
2.4	The Training Objective	9
2.5	The Posterior Distribution	9
2.6	The Reverse Process	10
2.7	The Learned Reverse Process	10
2.8	Parameterization Choice	10
3	Object Detection with Diffusion Models	10
3.1	The Central Idea	10
3.2	Comparison with Image Diffusion	10
3.3	Conditional Diffusion in Detection	10
4	DiffusionDet: Architecture	11
4.1	The Two-Part Architecture	11
4.2	The Image Encoder	11
4.3	The Detection Decoder	12
4.4	What Are the Proposal Boxes?	12
4.5	RoI Feature Extraction	12
4.6	The Detection Head	13
4.7	Training Process	13
4.8	GT Box Padding	13
4.9	Four Padding Strategies	14
4.10	Box Corruption (Forward Diffusion)	14
4.11	Simple Numerical Intuition	15
4.12	Why Scale the Box Coordinates?	15
4.13	The Training Prediction	15
4.14	The Prediction Count	16
4.15	The Matching Problem	16
4.16	Multiple Predictions per GT	17
4.17	Inference Process	17
4.18	The Key Inference Process	17
4.19	What Does DDIM Do?	18
4.20	Why Not Directly Feed Prediction Back?	18
4.21	Box Renewal Mechanism	18
4.22	Why Box Renewal Makes Sense	19
4.23	Complete Inference Algorithm	20
4.24	One Complete Example	20
	4.24.1 Step 1	20
	4.24.2 Step 2	21
	4.24.3 Step 3	21
5	DiffusionDet vs DETR: The Key Difference	22
5.1	DETR	22
5.2	DiffusionDet	22
5.3	Flexible Usage Property	22
5.4	Changing the Number of Iterations	23
5.5	The Deepest Difference	23

6	Dynamic Number of Boxes Property	23
6.1	The Claim	23
6.2	Why This is Useful	23
6.3	DETR's Problem	24
6.4	Why DETR Can't Simply Use 1000 Queries	24
6.5	How the Authors Tested DETR with More Queries	25
6.6	Why Cloning Doesn't Give New Learned Queries	25
6.7	What Happens Because of This?	25
6.8	Why NMS is Added to DETR	26
6.9	DETR Performance with More Queries	26
6.10	The "Concat Random" Strategy	26
6.11	Why This is Even Worse for DETR	27
6.12	The Result	27
6.13	DiffusionDet Performance with More Boxes	27
7	Iterative Evaluation	27
7.1	What is Iterative Evaluation?	27
7.2	What One Iteration Means	28
7.3	What Does $S@500$ Mean?	28
7.4	Results	28
7.5	Why Fewer Boxes Benefit More from Iterations	29
7.6	Two Ways to Improve Inference	29
7.7	Example of Speed-Accuracy Tradeoff	29
7.8	Comparing Iterative Evaluation Across Models	29
7.9	Without Ensemble	30
7.10	With Ensemble	30
8	Zero-Shot Transfer	30
8.1	What is Zero-Shot Transfer?	30
8.2	Why CrowdHuman is Interesting	30
8.3	Results	31
9	Diff3Det: 3D Object Detection	31
9.1	Introduction to Diff3Det	31
9.2	3D Object Detection Background	31
9.3	Point-Based vs Voxel-Based Methods	31
9.4	The Problem with Existing Methods	31
9.5	The Diff3Det Pipeline	32
9.6	BEV Box Representation	32
9.7	Constructing Initial Boxes x_0	32
9.8	Signal Scaling Factor	32
9.9	Forward Diffusion in Diff3Det	33
9.10	The Resampling Problem	33
9.11	Proposal Quality	33
10	Correlation Coefficient on Size	33
10.1	The Problem	33
10.2	The Solution: Correlation Coefficient	33
10.3	The Mathematical Formulation	33

10.4	Scaling the Random Vectors	34
10.5	Result	34
11	Dynamic Time Step	34
11.1	The Problem	34
11.2	The Solution: Sine Schedule	34
11.3	The Mathematical Formulation	34
11.4	Hyperparameter Values	34
12	Loss Function	35
12.1	Bipartite Matching	35
12.2	Definitions	35
12.3	Matching Cost	35
12.4	Optimal Assignment	35
12.5	Training Loss	35
12.6	Loss Weights	35
13	Inference Phase in Diff3Det	36
13.1	Overview	36
13.2	The Sampling Formula	36
13.3	Parameter Definitions	36
13.4	Sampling Steps	36
13.5	Post-Processing	36
14	Experiments and Results	36
14.1	Dataset	36
14.2	Implementation Details	37
14.2.1	Voxelization Range	37
14.2.2	Voxel Size	37
14.2.3	Data Augmentation	37
14.2.4	Training Configuration	37
14.3	Main Results	37
14.4	Multi-Step Sampling	37
15	Ablation Studies	38
15.1	Proposed Components	38
15.1.1	Baseline	38
15.1.2	Adding Box Corruption	38
15.1.3	Resampling Operation	38
15.1.4	Size Correlation Strategy	38
15.1.5	Dynamic Time Step	38
15.2	Sampling Steps	38
15.3	Hyperparameters	39
15.3.1	Signal Scale	39
15.3.2	Proposal Number	39
15.3.3	Resampling Parameter η	39

16 Limitations and Conclusion	39
16.1 Limitations of Diff3Det	39
16.2 Conclusion	40
17 Summary of Key Equations	40
18 The Big Picture	41
18.1 DiffusionDet Training	41
18.2 DiffusionDet Inference	41
18.3 Diff3Det Training	41
18.4 Diff3Det Inference	41
19 Additional Experimental Details and Results	41
19.1 DiffusionDet: Additional Experiments	41
19.1.1 Dynamic Number of Boxes: Training vs Inference Matching . . .	41
19.1.2 Running Time vs. Accuracy	42
19.1.3 Full Tuning on CrowdHuman	42
19.1.4 Random Seed Stability	43
19.1.5 Sampling Strategy: DDIM and Box Renewal	43
19.2 Diff3Det: Additional Experimental Details	43
19.2.1 Qualitative Results	43
19.2.2 AP40 Metrics	43
19.2.3 Comparison with Multi-Modal Methods	44
19.2.4 Dynamic Head Extension to 3D	44
19.2.5 GT Sampling Augmentation	44
19.2.6 AP11 vs AP40 Difference	44
19.2.7 Detailed Component Ablation for Diff3Det	45
19.2.8 Multi-Step Sampling Analysis	45
19.3 Summary of Key Experimental Findings	46

1 Introduction: The Evolution of Object Detection

1.1 Traditional Object Detection

Traditional object detection pipelines often contain hand-designed steps, for example:

- Manually defining anchor boxes
- Selecting proposals
- Using NMS (Non-Maximum Suppression)
- Designing rules for how candidate objects are generated

1.2 Empirical Object Priors

Empirical object priors means using predefined assumptions about where objects are likely to be and what their typical shapes/sizes are, based on observations from the training data.

Examples include:

- Anchor boxes $\rightarrow$ predefined widths, heights, and aspect ratios
- Object proposals $\rightarrow$ candidate regions generated using hand-designed or learned heuristics
- Typical object sizes, aspect ratios, and locations observed in the dataset

For example, if training data shows that many objects are roughly:

$$w = 40, \quad h = 80 \tag{1}$$

a detector might create an anchor/prior around that size.

Empirical object priors = predefined/observed assumptions about object candidates

(2)

1.3 The Evolution of Object Candidates

The paper says object candidates evolved roughly like this:

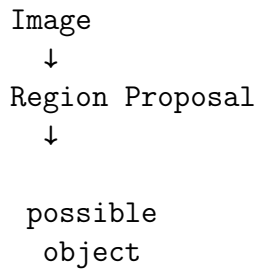
Sliding windows
 $\downarrow$
Region proposals
 $\downarrow$
Anchor boxes
 $\downarrow$
Reference points
 $\downarrow$
Learnable object queries
 $\downarrow$
Diffusion-based random boxes

1.4 Sliding Windows

Older methods used sliding windows. Imagine taking a fixed-size box and moving it across the image. At every position, the detector asks: "Is there an object inside this window?" The windows are hand-designed.

1.5 Region Proposals

Then came methods such as R-CNN / Faster R-CNN. Instead of checking every possible window, the system generates a set of region proposals.



1.6 Anchor Boxes

Then detectors such as Faster R-CNN/SSD/YOLO-style approaches used anchor boxes. At predefined positions, you place boxes with predefined sizes/aspect ratios. The network predicts:

- Is there an object here?
- What class is it?
- How should I shift/resize this anchor to fit the object?

So the network performs classification + regression relative to these predefined anchors.

1.7 Reference Points

Another approach is reference points. Instead of starting with a whole predefined box, you can start from a point. The network then predicts where the object is relative to that reference.

1.8 Learnable Object Queries

ETR [10] proposes learnable object queries to eliminate hand-designed components and build a fully end-to-end object detection pipeline.

An object query is essentially a learned vector that asks the network: "Is there an object corresponding to me, and if so, what are its class and location?"

The queries are learned during training rather than being manually designed.

So the overall idea is:

Traditional detector:

Image $\rightarrow$ features $\rightarrow$ hand-designed proposals/anchors $\rightarrow$ detection $\rightarrow$ NMS

Query-based end-to-end detector:

Image $\rightarrow$ features $\rightarrow$ learnable object queries $\rightarrow$ object predictions

1.9 The Core Change

The evolution described is:

$$\boxed{\text{Hand-designed object candidates} \rightarrow \text{learned object candidates}} \quad (3)$$

And this is why the authors say modern detection is moving toward a more end-to-end detection pipeline.

1.10 Object Detection Definition

The learning objective of object detection is input-target pairs (x, b, c) , where:

- x = input image
- b, c = a set of bounding boxes and category labels for objects in the image x

More specifically, the i -th box in the set is formulated as:

$$\boxed{b_i = (c_i^x, c_i^y, w_i, h_i)} \quad (4)$$

where:

- (c_i^x, c_i^y) = center coordinates of the bounding box
- (w_i, h_i) = width and height of that bounding box

2 Diffusion Models: Background

2.1 What is a Diffusion Model?

Diffusion models [38,84,85,87] are a class of likelihood-based models inspired by nonequilibrium thermodynamics [87,88]. These models define a Markovian chain of diffusion forward process by gradually adding noise to sample data.

2.2 The Forward Process

The forward noise process is defined as:

$$\boxed{q(z_t|z_0) = \mathcal{N}(z_t|\sqrt{\bar{\alpha}_t}z_0, (1 - \bar{\alpha}_t)I)} \quad (5)$$

This is Equation (1).

which transforms data sample z_0 to a latent noisy sample z_t for $t \in \{0, 1, \dots, T\}$ by adding noise to z_0 .

$$\bar{\alpha}_t := \prod_{s=0}^t \alpha_s = \prod_{s=0}^t (1 - \beta_s) \quad (6)$$

and β_s represents the noise variance schedule [38].

2.3 Reparameterization

As noted by Ho et al. [38], we can directly sample data z_t at an arbitrary timestep t without the need of applying q repeatedly:

$$q(z_t|z_0) := \mathcal{N}(z_t; \sqrt{\bar{\alpha}_t}z_0, (1 - \bar{\alpha}_t)I) \quad (7)$$

This is Equation (5).

Equivalently:

$$z_t = \sqrt{\bar{\alpha}_t}z_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \quad \epsilon \in \mathcal{N}(0, I) \quad (8)$$

This is Equation (6).

2.4 The Training Objective

During training, a neural network $f_\theta(z_t, t)$ is trained to predict z_0 from z_t by minimizing the training objective with ℓ_2 loss [38]:

$$L_{\text{train}} = \frac{1}{2} \|f_\theta(z_t, t) - z_0\|^2 \quad (9)$$

This is Equation (2).

2.5 The Posterior Distribution

Based on Bayes' theorem, it is found that the posterior $q(z_{t-1}|z_t, z_0)$ is a Gaussian distribution as well:

$$q(z_{t-1}|z_t, z_0) = \mathcal{N}(z_{t-1}; \tilde{\mu}(z_t, z_0), \tilde{\beta}_t I) \quad (10)$$

This is Equation (7).

where:

$$\tilde{\mu}_t(z_t, z_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} z_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} z_t \quad (11)$$

This is Equation (8).

and:

$$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t \quad (12)$$

This is Equation (9).

are the mean and variance of this Gaussian distribution.

2.6 The Reverse Process

We could get a sample from $q(z_0)$ by first sampling from $q(z_T)$ and running the reversing steps $q(z_{t-1}|z_t)$ until z_0 .

Besides, the distribution of $q(z_T)$ is nearly an isotropic Gaussian distribution with a sufficiently large T and reasonable schedule of β_t ($\beta_t \rightarrow 0$), which making it trivial to sample $z_T \sim \mathcal{N}(0, I)$.

2.7 The Learned Reverse Process

Since calculating $q(z_{t-1}|z_t)$ exactly should depend on the entire data distribution, we could approximate $q(z_{t-1}|z_t)$ using a neural network, which is optimized to predict a mean μ_θ and a diagonal covariance matrix Σ_θ :

$$\boxed{p_\theta(z_{t-1}|z_t) := \mathcal{N}(z_{t-1}; \mu_\theta(z_t, t), \Sigma_\theta(z_t, t))} \quad (13)$$

This is Equation (10).

2.8 Parameterization Choice

Instead of directly parameterizing $\mu_\theta(z_t, t)$, Ho et al. [38] found learning a network $f_\theta(z_t, t)$ to predict the ϵ or z_0 from Equation (6) worked best.

DiffusionDet chooses to predict z_0 directly.

3 Object Detection with Diffusion Models

3.1 The Central Idea

The central idea of DiffusionDet is:

Instead of directly predicting the final object boxes in one shot, start with noisy/random boxes and progressively refine them into correct object boxes.

$$\boxed{\text{Random / noisy boxes} \rightarrow \text{refinement step} \rightarrow \text{better boxes} \rightarrow \text{refinement step} \rightarrow \dots \rightarrow \text{final object boxes}} \quad (14)$$

3.2 Comparison with Image Diffusion

Image diffusion:

random image noise $\rightarrow$ denoising $\rightarrow$ image

DiffusionDet:

random/noisy boxes $\rightarrow$ box denoising $\rightarrow$ object boxes

3.3 Conditional Diffusion in Detection

In DiffusionDet, the condition is the image features:

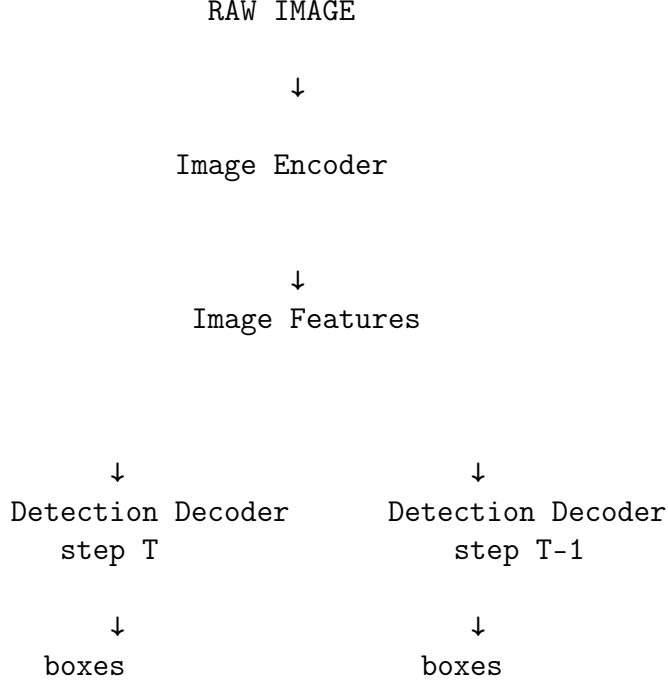
$$\boxed{\text{random/noisy boxes} + \text{image features} \rightarrow \text{denoised object boxes}} \quad (15)$$

4 DiffusionDet: Architecture

4.1 The Two-Part Architecture

The detector is divided into two parts:

1. **Image Encoder:** Extracts image features once
2. **Detection Decoder:** Runs repeatedly for denoising



4.2 The Image Encoder

The image encoder takes the raw image x and converts it into high-level feature maps:

$$F = \text{ImageEncoder}(x) \quad (16)$$

For example, DiffusionDet can use:

- ResNet
- Swin Transformer

as the backbone.

Then it uses an FPN.

Conceptually:

Image $\rightarrow$ ResNet/Swin $\rightarrow$ FPN $\rightarrow$ multi-scale feature maps

(17)

4.3 The Detection Decoder

The detection decoder takes:

$$\boxed{\text{proposal boxes} + \text{image features} + \text{timestep}} \quad (18)$$

and produces predictions.

In mathematical form:

$$\boxed{\hat{z}_0 = f_\theta(z_t, F, t)} \quad (19)$$

where:

- z_t = noisy boxes at timestep t
- F = image features
- t = current diffusion timestep
- f_θ = detection decoder
- $\hat{z}_0$ = predicted clean boxes

4.4 What Are the Proposal Boxes?

In DiffusionDet, the input is actually a bounding box.

For example, a box can be represented as:

$$(x, y, w, h) \quad (20)$$

So one proposal might be:

$$z_1 = (0.4, 0.5, 0.2, 0.3) \quad (21)$$

and another:

$$z_2 = (0.7, 0.2, 0.1, 0.15) \quad (22)$$

These are candidate box coordinates.

4.5 RoI Feature Extraction

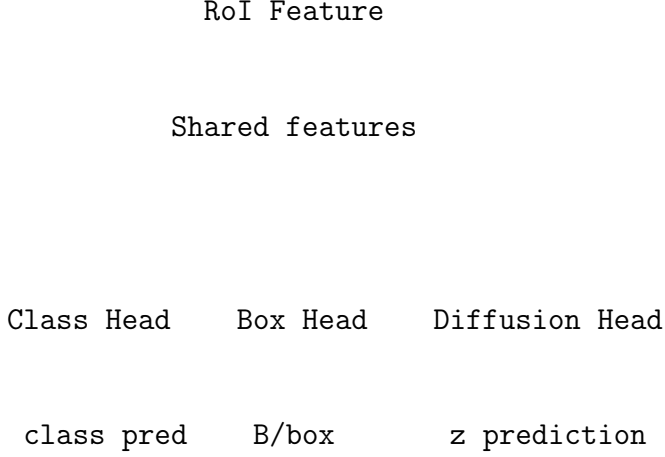
The detection decoder takes as input a set of proposal boxes to crop RoI-feature from feature map generated by image encoder.

proposal box $\rightarrow$ crop corresponding feature region $\rightarrow$ RoI feature

Then that RoI feature is sent to the detection head.

4.6 The Detection Head

The detection head has multiple branches:



Each branch produces:

$$\text{candidate box} \rightarrow \begin{cases} \text{class prediction} \\ \text{box prediction} \\ \text{diffusion prediction} \end{cases} \quad (23)$$

So for candidate i :

$$B_i \rightarrow F_i \rightarrow \begin{cases} p_i & \text{class} \\ \Delta B_i & \text{box refinement} \\ \hat{z}_{0,i} & \text{clean-box prediction} \end{cases} \quad (24)$$

4.7 Training Process

The training process starts with ground-truth boxes and adds noise.

Suppose an image has 3 ground-truth objects:

- $B_1 \rightarrow \text{person}$
- $B_2 \rightarrow \text{car}$
- $B_3 \rightarrow \text{dog}$

But the model is configured with $N_{\text{train}} = 10$.

So the model needs 10 boxes as input. But there are only 3 ground-truth boxes.

4.8 GT Box Padding

The paper says:

"we first pad some extra boxes to original ground truth boxes such that all boxes are summed up to a fixed number N_{train} ."

So:

Actual GT:

B1 = person

B2 = car

B3 = dog

Need 10 boxes:

B1 = person

B2 = car

B3 = dog

B4 = extra

B5 = extra

B6 = extra

B7 = extra

B8 = extra

B9 = extra

B10 = extra

4.9 Four Padding Strategies

The paper tests four ways of creating extra boxes:

1. **Repeat GT boxes:** Simply repeat existing GT boxes until reaching N_{train}
2. **Gaussian random boxes:** Generate random boxes using a Gaussian distribution
3. **Uniform random boxes:** Generate extra boxes using a uniform distribution
4. **Whole-image boxes:** Every extra box is initialized to approximately the size of the entire image

$$\boxed{\text{Gaussian random boxes work best}} \quad (25)$$

4.10 Box Corruption (Forward Diffusion)

Now we have padded clean boxes. The paper adds Gaussian noise:

$$\boxed{z_t = \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon} \quad (26)$$

where:

- z_0 = clean box
- z_t = noisy box at timestep t
- $\epsilon \sim \mathcal{N}(0, I)$
- $\bar{\alpha}_t$ controls how much original box information remains

4.11 Simple Numerical Intuition

Suppose the clean box is:

$$z_0 = (0.5, 0.5, 0.2, 0.3) \quad (27)$$

At a relatively small timestep:

$$z_t = (0.48, 0.52, 0.22, 0.29) \quad (28)$$

At a much noisier timestep:

$$z_t = (0.72, 0.31, 0.08, 0.51) \quad (29)$$

At an extremely noisy timestep, it can be almost random.

4.12 Why Scale the Box Coordinates?

The paper also says "the ground truth box coordinates need to be scaled as well" because the signal-to-noise ratio matters.

Usually bounding boxes are normalized, for example:

$$x, y, w, h \in [0, 1] \quad (30)$$

The authors found that simply using these values directly wasn't optimal for diffusion. So they transform the coordinates:

$$\boxed{z_0 = (2b - 1) \times \text{scale}} \quad (31)$$

where b represents normalized box coordinates.

4.13 The Training Prediction

After corruption, the detector receives:

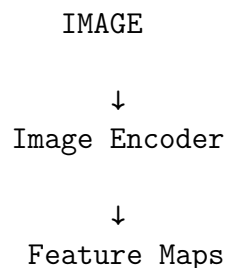
$$z_t, \quad F, \quad t \quad (32)$$

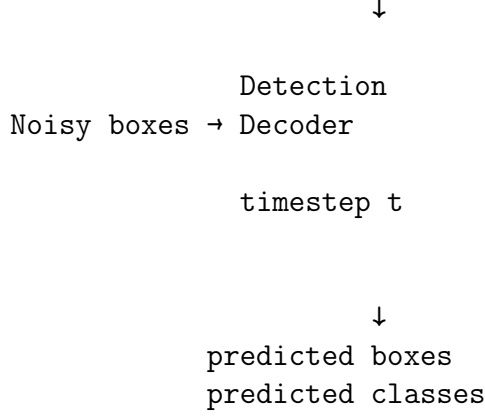
and predicts:

$$\hat{z}_0 \quad (33)$$

plus class predictions.

So:

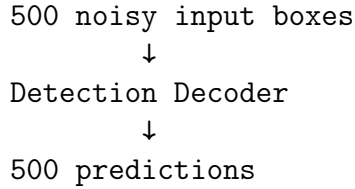




4.14 The Prediction Count

The detection detector takes as input N_{train} corrupted boxes and predicts N_{train} predictions.

So if $N_{\text{train}} = 500$:



Each input proposal box has its own corresponding prediction pathway:

$$\boxed{
\begin{array}{l}
z_1 \rightarrow \text{Decoder} \rightarrow \text{prediction 1} \\
z_2 \rightarrow \text{Decoder} \rightarrow \text{prediction 2} \\
z_3 \rightarrow \text{Decoder} \rightarrow \text{prediction 3} \\
\vdots \\
z_{500} \rightarrow \text{Decoder} \rightarrow \text{prediction 500}
\end{array}
} \tag{34}$$

Each prediction contains:

$$\boxed{\text{class} + \text{box}} \tag{35}$$

4.15 The Matching Problem

Suppose $N_{\text{train}} = 500$ but the image contains only 3 objects. We have 500 predictions but only 3 ground-truth objects.

This is where set prediction loss / optimal transport assignment comes in.

Predictions:

P1, P2, P3, ..., P500

Ground truth:

GT1 = person

GT2 = car

GT3 = dog

The model finds which predictions are the best matches to each GT.

4.16 Multiple Predictions per GT

DiffusionDet assigns multiple predictions to each ground truth:

"We assign multiple predictions to each ground truth by selecting the top k predictions with the least cost by an optimal transport assignment method."

For example:

GT person
↓
P87 P102 P145

These are the top- k predictions having low matching cost with that ground truth.

This helps train the diffusion detector because many noisy proposals can converge toward the same object.

4.17 Inference Process

During inference, there are no ground-truth boxes. So where do the initial boxes come from?

The paper says:

"directly sampled from Gaussian distribution at evaluation stage."

So:

$$\boxed{z_T \sim \mathcal{N}(0, I)} \tag{36}$$

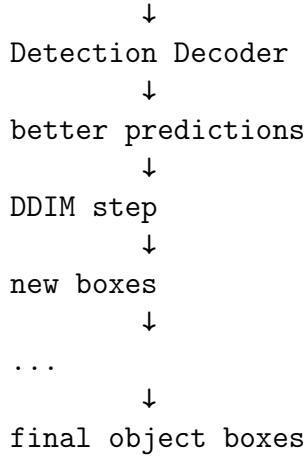
In simple terms:

Random Gaussian noise
↓
random boxes

4.18 The Key Inference Process

Suppose we start with 1000 random boxes:

1000 random boxes
↓
Detection Decoder
↓
rough predictions
↓
DDIM step
↓
new boxes



4.19 What Does DDIM Do?

The detector predicts an estimate of the clean box $\hat{z}_0$ from the current noisy box z_t .

Then DDIM uses that information to estimate what the box should look like at the next timestep:

$$z_{t-1} \tag{37}$$

So:

$$\boxed{z_t \rightarrow \hat{z}_0 \rightarrow z_{t-1}} \tag{38}$$

Then the process repeats.

4.20 Why Not Directly Feed Prediction Back?

The authors mention that you could do:

prediction $\rightarrow$ directly feed prediction back $\rightarrow$ next iteration

But that doesn't match the diffusion training process very well.

During training, the model learns to deal with boxes generated by the specific Gaussian corruption process. Therefore, at inference, they use DDIM to generate the next noisy state in a way that is consistent with that learned diffusion process.

So:

Training:

clean box $\rightarrow$ Gaussian corruption $\rightarrow$ noisy box

Inference:

random noisy box $\rightarrow$ DDIM denoising $\rightarrow$ less noisy box

4.21 Box Renewal Mechanism

This is another subtle but important idea.

After one denoising step:

Box 1 → good → near a person
 Box 2 → good → near a car
 Box 3 → good → near dog
 Box 4 → bad → random location
 Box 5 → bad → random location
 ...

The good boxes are useful. But what about the bad ones?

The paper calls them undesired predictions. If you keep feeding those bad boxes forward, they may continue to be useless.

So DiffusionDet says: Let's replace the bad boxes with new random boxes.

Good:

B1 → person
 B2 → car
 B3 → dog

Bad:

B4 → random
 B5 → random
 B6 → random

After box renewal:

B1 → keep
 B2 → keep
 B3 → keep

B4 → NEW RANDOM BOX
 B5 → NEW RANDOM BOX
 B6 → NEW RANDOM BOX

So:

$$\boxed{\text{good predictions} \rightarrow \text{preserve}} \quad (39)$$

$$\boxed{\text{bad predictions} \rightarrow \text{replace with random boxes}} \quad (40)$$

4.22 Why Box Renewal Makes Sense

Because the training distribution contains boxes generated by a known corruption process. If an inference box becomes some weird arbitrary prediction that is not representative of the training corruption distribution, feeding it forward can be harmful.

Randomly replacing it brings the input distribution closer to what the model saw during training.

That's what the paper means by:

"To make inference better align with training"

4.23 Complete Inference Algorithm

The pseudo-code says:

```
images
  ↓
image_encoder(images)
  ↓
feature maps
(once)
```

Then:

```
pb_t = Gaussian random boxes
(e.g.,  $N_{\text{eval}} = 1000$ )
pbt  $\sim \mathcal{R}^{B \times 1000 \times 4}$ 
```

Then repeatedly:

```
current noisy boxes
  ↓
Detection Decoder
  ↓
predicted clean boxes
  ↓
DDIM
  ↓
next noisy boxes
  ↓
box renewal
  ↓
repeat
```

4.24 One Complete Example

Let's say $N_{\text{eval}} = 5$ and the image actually contains:

- person
- car
- dog

We start with 5 random boxes:

R_1, R_2, R_3, R_4, R_5

4.24.1 Step 1

The decoder sees R_1, R_2, R_3, R_4, R_5 + image features + timestep and predicts:

P1 → random
P2 → somewhat near person
P3 → random
P4 → near car
P5 → random

After DDIM + renewal:

B1 → new random
B2 → person-ish
B3 → new random
B4 → car-ish
B5 → new random

4.24.2 Step 2

Feed them back:

B1, B2, B3, B4, B5

The decoder might produce:

P1 → random
P2 → person
P3 → random
P4 → car
P5 → dog-ish

Renew again:

random
person
random
car
dog

4.24.3 Step 3

The boxes become increasingly accurate:

person → accurate
car → accurate
dog → accurate

5 DiffusionDet vs DETR: The Key Difference

5.1 DETR

DETR starts with N learnable queries.

For example: 300 learned queries.

Q1 $\rightarrow$ prediction
Q2 $\rightarrow$ prediction
...
Q300 $\rightarrow$ prediction

The queries are learned parameters.

5.2 DiffusionDet

DiffusionDet starts with N boxes during inference:

random box 1
random box 2
...
random box N

Then repeatedly refines them:

random boxes $\rightarrow$ denoise $\rightarrow$ better boxes $\rightarrow$ denoise $\rightarrow$ better boxes $\rightarrow$...

So:

$$\boxed{\text{DETR: learned queries}} \tag{41}$$

$$\boxed{\text{DiffusionDet: noisy/random boxes}} \tag{42}$$

5.3 Flexible Usage Property

This explains the "flexible usage" property.

Suppose during training: $N_{\text{train}} = 500$.

At inference you can use:

$$N_{\text{eval}} = 100, \quad 1000, \quad 4000, \dots \tag{43}$$

Why? Because these aren't 100, 1000, or 4000 learned query parameters. They are just random initial boxes.

The detector has learned:

"Given a noisy box and image features, refine that box toward an object."

Therefore you can provide more or fewer boxes.

5.4 Changing the Number of Iterations

Suppose you use 5 diffusion steps. You get one speed/accuracy trade-off.

If you use 10 steps, you get another.

If you use 20 steps, you get another.

Few boxes + few steps → very fast → possibly lower accuracy

Many boxes + many steps → slower → potentially better accuracy

5.5 The Deepest Difference

$$\boxed{\text{DETR learns a fixed set of query representations}} \quad (44)$$

whereas:

$$\boxed{\text{DiffusionDet learns a box-refinement/denoising process}} \quad (45)$$

Therefore:

DETR:

300 queries → naturally 300 prediction slots

DiffusionDet:

300 training boxes 300 permanently fixed box slots

$N_{\text{train}} = 300$ but N_{eval} can vary

6 Dynamic Number of Boxes Property

6.1 The Claim

The authors want to show that DiffusionDet has a special property:

After training, you don't have to keep the number of boxes fixed.

Suppose DiffusionDet is trained with $N_{\text{train}} = 300$.

After training, you can run it with:

$$N_{\text{eval}} = 50, \quad 100, \quad 300, \quad 500, \quad 4000 \quad (46)$$

And you use the same trained network parameters.

$$\boxed{\text{Train once} \rightarrow \text{use different numbers of boxes at inference}} \quad (47)$$

6.2 Why This is Useful

Imagine two images:

Image A - sparse: only a few objects

Image B - crowded: hundreds of objects

For a sparse image, you may not need thousands of candidate boxes. For a crowded image, having more candidate boxes can help.

So ideally:

sparse image $\rightarrow$ fewer boxes
crowded image $\rightarrow$ more boxes

DiffusionDet allows this without retraining.

6.3 DETR’s Problem

DETR uses object queries. Suppose $N_{\text{queries}} = 300$.

That means DETR has 300 learned object queries.

$$\boxed{300 \text{ queries} \rightarrow 300 \text{ prediction slots}} \quad (48)$$

Does 300 queries mean there must be 300 objects? No.

Suppose an image contains only 7 objects. You still have 300 queries.

The output might look like:

Query 1 $\rightarrow$ dog
Query 2 $\rightarrow$ car
Query 3 $\rightarrow$ person
...
Query 8 $\rightarrow$ background/no object
...
Query 300 $\rightarrow$ background/no object

So:

$$\boxed{300 \text{ queries} \neq 300 \text{ actual objects}} \quad (49)$$

but:

$$\boxed{300 \text{ queries} \approx 300 \text{ maximum prediction slots}} \quad (50)$$

6.4 Why DETR Can’t Simply Use 1000 Queries

Because the original DETR model was trained with 300 learned queries. Those queries are actual learned parameters.

$$Q = \{q_1, q_2, \dots, q_{300}\} \quad (51)$$

The important point is: the queries themselves are learned parameters.

So after training, DETR doesn’t magically have:

$$q_{301}, q_{302}, \dots, q_{1000} \quad (52)$$

They were never part of the trained model.

6.5 How the Authors Tested DETR with More Queries

They had to create a workaround. They call one strategy:

$$\boxed{\text{clone}} \tag{53}$$

Suppose $N_{\text{train}} = 300$ and they want $N_{\text{eval}} = 1000$.

They already have $q_1, q_2, \dots, q_{300}$.

They need 700 additional queries. So they copy some existing queries:

$q_1, q_2, \dots, q_{300}, q_1, q_2, \dots, q_{700}$

until they have 1000.

$$\boxed{300 \text{ original queries} + 700 \text{ cloned queries} = 1000} \tag{54}$$

6.6 Why Cloning Doesn't Give New Learned Queries

Because the new queries aren't actually new learned representations.

Suppose:

$$q_1 = [0.2, 0.7, 0.4, \dots] \tag{55}$$

You clone it:

$$q_{301} = q_1 \tag{56}$$

So you're not getting 700 new independently learned queries. You're just copying existing ones.

6.7 What Happens Because of This?

Suppose:

$$q_1 \rightarrow \text{predicts a box around a dog} \tag{57}$$

If you clone it:

$$q_{301} = q_1 \tag{58}$$

then q_{301} has the same starting representation. Therefore it can produce a prediction very similar to the prediction produced by q_1 .

$q_1 \rightarrow \text{Dog at approximately } (100, 200)$
 $q_{301} = q_1 \rightarrow \text{Dog at approximately } (100, 200)$

You may therefore get duplicate or highly similar detections.
 That's why the authors say:

"cloned queries will produce similar detection results as the original queries."

6.8 Why NMS is Added to DETR

Because cloning creates similar predictions. Suppose the output contains:

Box 1 → dog, confidence 0.95

Box 2 → dog, confidence 0.93

Box 3 → dog, confidence 0.91

and all three boxes overlap heavily. They may actually correspond to the same dog. So they use:

$$\boxed{\text{NMS} = \text{Non-Maximum Suppression}} \quad (59)$$

to remove redundant overlapping predictions.

6.9 DETR Performance with More Queries

They trained DETR with 300 queries. At 300 queries:

$$38.8 \text{ AP} \quad (60)$$

Then they increase the number of queries using cloning.

Queries at inference	DETR AP
50	31.0
100	34.9
300	38.8
500	38.4
1000	38.4
2000	38.4
4000	38.4

At 300 → 500 → 1000 → 2000 → 4000, the performance doesn't improve. It actually drops slightly: 38.8 → 38.4.

6.10 The "Concat Random" Strategy

Instead of cloning existing queries, they create:

$$N_{\text{eval}} - N_{\text{train}} \quad (61)$$

randomly initialized queries.

Suppose $N_{\text{train}} = 300$ and $N_{\text{eval}} = 1000$.

They already have 300 learned queries. They add:

$$1000 - 300 = 700 \quad (62)$$

random queries.

$$\boxed{300 \text{ learned queries} + 700 \text{ random queries} = 1000} \quad (63)$$

6.11 Why This is Even Worse for DETR

Because those extra 700 queries are not trained. The original queries $q_1, \dots, q_{300}$ have been optimized during training.

But the additional queries $q_{301}, \dots, q_{1000}$ are randomly initialized.

The model never learned how to use these additional query representations properly.

Therefore, as you increase $N_{\text{eval}} - N_{\text{train}}$, you are adding more and more poorly trained/random queries. Performance gets worse.

6.12 The Result

At $N_{\text{eval}} = 300$, DETR has 38.8 AP.

But at $N_{\text{eval}} = 4000$ with concat-random:

$$26.4 \text{ AP} \tag{64}$$

That's a huge decrease:

$$38.8 - 26.4 = 12.4 \tag{65}$$

So:

26.4 AP = 12.4 AP lower than best

(66)

6.13 DiffusionDet Performance with More Boxes

300 boxes $\rightarrow$ 45.8 AP

4000 boxes $\rightarrow$ 46.8 AP

So:

+1.0 AP

(67)

from increasing boxes from 300 to 4000.

300 $\rightarrow$ 500 $\rightarrow$ 1000 $\rightarrow$ 2000 $\rightarrow$ 4000

45.8 $\rightarrow$ 46.3 $\rightarrow$ 46.7 $\rightarrow$ 46.8 $\rightarrow$ 46.8

So performance improves and then saturates.

more proposal boxes $\rightarrow$ more opportunities $\rightarrow$ better AP

(68)

7 Iterative Evaluation

7.1 What is Iterative Evaluation?

This is separate from the number of boxes. There are actually two independent knobs:

1. **Number of boxes:** $N_{\text{eval}} = 100, 300, 500, 1000, \dots$
2. **Number of refinement iterations:** $S = 1, 2, 3, \dots, 8$

You can change either one without retraining.

7.2 What One Iteration Means

Suppose we start with noisy/random boxes $B^{(0)}$.

DiffusionDet processes them and refines them:

$$B^{(0)} \rightarrow B^{(1)} \tag{69}$$

That's one refinement step.

If we do another:

$$B^{(1)} \rightarrow B^{(2)} \tag{70}$$

Then:

$$B^{(0)} \rightarrow B^{(1)} \rightarrow B^{(2)} \tag{71}$$

For 8 iterations:

$$\boxed{B^{(0)} \rightarrow B^{(1)} \rightarrow B^{(2)} \rightarrow \dots \rightarrow B^{(8)}} \tag{72}$$

7.3 What Does $S@500$ Mean?

$S@500$ means:

Evaluate using 500 boxes, while changing the number of refinement iterations S .

So:

$S@500$, $S = 1$: 500 boxes + 1 refinement step

$S@500$, $S = 8$: 500 boxes + 8 refinement steps

7.4 Results

For 100 boxes:

1 step: 41.9

8 steps: 46.1

That's:

$$46.1 - 41.9 = 4.2 \tag{73}$$

AP improvement.

$$\boxed{+4.2 \text{ AP}} \tag{74}$$

just by allowing more refinement iterations.

7.5 Why Fewer Boxes Benefit More from Iterations

DiffusionDet with fewer random boxes tends to achieve more substantial gains with refinement.

Think about 100 boxes. Initially, 100 random boxes may not give you excellent coverage of the objects. But repeated refinement gives those boxes more opportunities to move toward useful locations.

So:

$$\boxed{100 \text{ boxes} \xrightarrow{\text{more refinement}} \text{much better boxes}} \quad (75)$$

With 500 boxes, you already have much better initial coverage. Therefore, additional refinement gives a smaller relative improvement.

7.6 Two Ways to Improve Inference

You can increase:

1. **Number of boxes:** $N_{\text{eval}} \uparrow \Rightarrow$ potentially better accuracy, but computational cost increases
2. **Refinement steps:** $S \uparrow \Rightarrow$ more refinement $\Rightarrow$ potentially better accuracy, but computation also increases

Therefore you can choose a speed-accuracy tradeoff.

7.7 Example of Speed-Accuracy Tradeoff

Very fast, lower AP: 100 boxes, $S = 1$

Better accuracy: 300 boxes, $S = 4$

Highest accuracy, more computation: 1000 boxes, $S = 8$

And none of these require retraining.

That's the meaning of:

"desired speed-accuracy trade-off without re-training"

7.8 Comparing Iterative Evaluation Across Models

Table 8 shows the comparison:

Table 1: Iterative Evaluation Results

Method	Step 1	Step 3	Step 5
DETR	42.03	42.00 (-0.03)	41.88 (-0.15)
Deformable DETR	44.46	43.45 (-1.01)	43.40 (-1.06)
Sparse R-CNN	45.02	1.32 (-43.70)	0.32 (-44.70)
DiffusionDet	45.80	45.74 (-0.06)	45.46 (-0.34)

7.9 Without Ensemble

They observe that if they simply use predictions from later steps without ensemble, performance generally decreases.

For DiffusionDet:

$$45.80 \rightarrow 45.74 \rightarrow 45.46 \quad (76)$$

For DETR:

$$42.03 \rightarrow 42.00 \rightarrow 41.88 \quad (77)$$

But Sparse R-CNN is disastrous:

$$45.02 \rightarrow 1.32 \rightarrow 0.32 \quad (78)$$

7.10 With Ensemble

Now they combine predictions from multiple steps and apply NMS.

For DiffusionDet, the result actually improves.

The paper states:

"5 steps gives 0.99 AP improvement over 1 step"

So if step 1 is 45.80, the ensemble of multiple steps can reach 46.79.

That's why they say:

"DiffusionDet turns performance down to up."

More refinement

↓

more predictions from different refinement stages

↓

ensemble them

↓

NMS removes duplicates

↓

better final detection

8 Zero-Shot Transfer

8.1 What is Zero-Shot Transfer?

The paper takes a model trained on COCO and applies it to CrowdHuman without fine-tuning.

8.2 Why CrowdHuman is Interesting

COCO has people, but CrowdHuman is specifically more crowded with people. So the model encounters a situation with many more closely packed objects.

This is exactly where having more candidate boxes can be useful.

8.3 Results

They take the already-trained DiffusionDet and simply increase N_{eval} from 300 to 2000.

They get:

$$\boxed{+5.3 \text{ AP}} \tag{79}$$

They also increase refinement $S : 1 \rightarrow 4$ and get:

$$\boxed{+4.8 \text{ AP}} \tag{80}$$

Again, no additional fine-tuning.

9 Diff3Det: 3D Object Detection

9.1 Introduction to Diff3Det

Diff3Det is a framework that explores the feasibility of generative models for 3D object detection.

The model adds Gaussian noise with a controlled variance schedule to the ground truth boxes in the training stage to obtain noisy boxes.

These noisy boxes are then used to extract Region of Interest (RoI) features from the BEV feature map, which does not need to set the manual anchor.

The detection decoder then incorporates these features and time planes to predict offsets between the noisy and ground truth boxes.

9.2 3D Object Detection Background

3D object detection is an essential task for achieving autonomous driving. It aims to regress the 3D bounding boxes and recognize the corresponding category from the point clouds.

Due to the intrinsic sparsity and irregularity of the derived point clouds, building high-accuracy LiDAR-based 3D object detection methods is challenging.

9.3 Point-Based vs Voxel-Based Methods

Mainstream approaches can be divided into two categories:

1. **Point-based methods:** [36,47] directly learn geometry from unstructured point clouds. However, they are challenging to handle large-scale point clouds scenarios due to high computation costs.
2. **Voxel-based methods:** [49,42,19,28,37,9] convert irregular raw point clouds into regular voxel grid format and implement efficient feature extraction on voxels by highly optimized 3D sparse convolution operations.

9.4 The Problem with Existing Methods

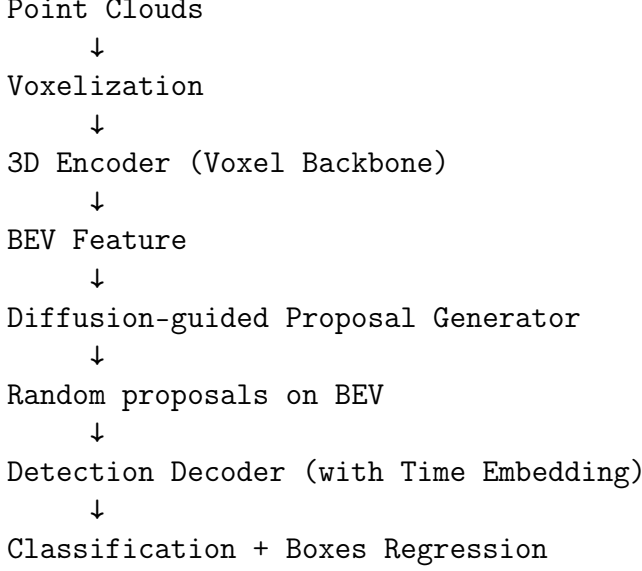
Existing voxel-based methods [42,49,19,28,44] still rely on empirical heuristics to set anchor sizes or center radii, which might not be an elegant strategy.

This leads to the question:

Is it feasible to recover predictive boxes from more succinct random boxes directly?

9.5 The Diff3Det Pipeline

The pipeline consists of:



9.6 BEV Box Representation

BEV (Bird’s Eye View) is an effective representation for 3D object detection. Therefore, Diff3Det uses the BEV boxes for the diffusion process.

The BEV box is represented as:

$$\boxed{(c_x, c_y, d_x, d_y, \theta)} \quad (81)$$

where:

- c_x, c_y = center coordinates
- d_x, d_y = dimensions (width and length)
- θ = orientation angle

9.7 Constructing Initial Boxes x_0

For constructing the initial boxes x_0 :

1. Repeat the ground truth boxes to N
2. Normalize them between 0 and 1

9.8 Signal Scaling Factor

A signal scaling factor controls the diffusion process’s signal-to-noise ratio (SNR) [7].

9.9 Forward Diffusion in Diff3Det

The corrupted boxes x_t are generated by adding Gaussian noise to x_0 , formulated as:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon \quad (82)$$

where:

- $\epsilon \sim \mathcal{N}(0, I_5)$
- $t = \text{randint}(1, T_{\max})$
- $\bar{\alpha}_t$ is the same as [39]
- The maximum time $T_{\max}$ is set to an integer (e.g., 1000)

9.10 The Resampling Problem

A proposal box with no point makes it tough to recover the target.

The paper adopts a resampling operation by calculating the number of points m in each proposal box.

$$\text{If } m < \eta, \text{ remove the boxes and resample random boxes} \quad (83)$$

Repeat this loop until every proposal box has at least η points in it.

9.11 Proposal Quality

The quality of the proposal boxes is the key to the success of Diff3Det. Simple ways to refine the proposal boxes are adopted.

10 Correlation Coefficient on Size

10.1 The Problem

It is clear that in the real world, there is a definite relationship between the width and length of a 3D detection box. However, the two independent random distributions of width and length will produce unrealistic proposals.

Therefore, it is inappropriate to consider the size (w, l) as two independent and identically distributed random variables.

10.2 The Solution: Correlation Coefficient

A correlation coefficient is introduced to restrict the box size for those resampled boxes and the boxes used during inference.

10.3 The Mathematical Formulation

$$W = \rho L + \sqrt{1 - \rho^2}X \quad (84)$$

where:

- $L, X \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1)$
- $\rho = \text{correlation coefficient (set to 0.8)}$

10.4 Scaling the Random Vectors

After generating the random vector (W, L) , they are scaled to the ranges $(0, w)$ and $(0, l)$ as the width and length of the proposals.

$$w = 8, \quad l = 5 \quad (85)$$

are set to satisfy the target.

10.5 Result

After the correlation coefficient constraint, the distribution of generated proposals is more correlated with ground truth.

11 Dynamic Time Step

11.1 The Problem

In the early training phase, recovering samples from seriously corrupted samples with high noise levels is difficult, which harms the final performance.

11.2 The Solution: Sine Schedule

A sine schedule is proposed to control the time step range, where the noise gradually increases in training.

11.3 The Mathematical Formulation

n is the total training epoch number, x is the current epoch index, and T is the maximum time to reach.

The maximum time on one training epoch $T_{\max}$ can be calculated as:

$$T_{\max} = \begin{cases} \frac{T}{\sin} \cos^{-1} \left(\frac{\omega}{T} \right) \sigma n x + \sin^{-1} \left(\frac{\omega}{T} \right), & x < \sigma n \\ T, & x \geq \sigma n \end{cases} \quad (86)$$

where:

- ω = hyperparameter controlling the initial time steps at the first epoch
- σ = hyperparameter controlling the training time-point to reach the maximum time T

11.4 Hyperparameter Values

The paper empirically sets:

$$\omega = 5, \quad \sigma = 0.5 \quad (87)$$

12 Loss Function

12.1 Bipartite Matching

Some methods [4,40] minimize the time-consuming post-processing operation by the bipartite matching. Therefore, Diff3Det extends the bipartite matching from 2D to 3D.

12.2 Definitions

Given:

- Ground truth set of objects: $y = \{y_i\}_{i=1}^M$
- Set of N predictions: $\hat{y} = \{\hat{y}_i\}_{i=1}^N$

12.3 Matching Cost

The matching cost is defined as follows:

$$C_{\text{match}} = \lambda_{\text{cls}} \cdot L_{\text{cls}} + \lambda_{\text{reg}} \cdot L_{\text{reg}} + \lambda_{\text{IoU}} \cdot L_{\text{BEV IoU}} \quad (88)$$

where:

- $\lambda_{\text{cls}}, \lambda_{\text{reg}}, \lambda_{\text{IoU}}$ = coefficients of each component
- L_{cls} = focal loss [24] of predicted classifications and ground truth category labels
- $L_{\text{reg}} = \ell_1$ loss between the normalized predicted boxes and ground truth boxes following [49]
- $L_{\text{BEV IoU}} = \text{BEV IoU loss following [4,40]}$

12.4 Optimal Assignment

$$C = \arg \min_{i \in M, j \in N} C_{\text{match}}(\hat{y}_i, y_j) \quad (89)$$

12.5 Training Loss

The training loss consists of classification, regression, and IoU, applied only to the matched pairs.

The IoU loss adopts the rotated 3D DIoU loss [48] denoted as L_{DIoU} .

$$L = \lambda_{\text{cls}} \cdot L_{\text{cls}} + \lambda_{\text{reg}} \cdot L_{\text{reg}} + \lambda_{\text{IoU}} \cdot L_{\text{DIoU}} \quad (90)$$

where the $\lambda_{\text{cls}}, \lambda_{\text{reg}}, \lambda_{\text{IoU}}$ represent the weight of corresponding loss, which is the same as the parameters in the matching cost.

12.6 Loss Weights

The paper sets:

$$\lambda_{\text{cls}} = 2, \quad \lambda_{\text{reg}} = 5, \quad \lambda_{\text{IoU}} = 2 \quad (91)$$

13 Inference Phase in Diff3Det

13.1 Overview

The inference procedure of Diff3Det is a denoising process from noise to object boxes.

Diff3Det progressively refines its predictions from boxes sampled in Gaussian distribution.

13.2 The Sampling Formula

The proposal boxes for the next step can be computed by the formula [39]:

$$x_{t-s} = \sqrt{\alpha_{t-s}} \left(\frac{x_t - \sqrt{1 - \alpha_t} \epsilon_{\theta}^{(t)}(x_t)}{\sqrt{\alpha_t}} \right) + \sqrt{1 - \alpha_{t-s} - \sigma_t^2} \cdot \epsilon_{\theta}^{(t)}(x_t) + \sigma_t \epsilon_t \quad (92)$$

where:

$$\sigma_t = \sqrt{\frac{1 - \alpha_t / \alpha_{t-s}}{(1 - \alpha_{t-s}) / (1 - \alpha_t)}} \quad (93)$$

13.3 Parameter Definitions

- x_t, x_{t-s} = proposal boxes in two adjacent steps
- $\epsilon_{\theta}^{(t)}(x_t)$ = predicted offsets by the decoder
- ϵ_t = Gaussian noises

13.4 Sampling Steps

The number of sampling steps is allowed to be equal to or higher than 1.

s is the starting time level (i.e., 1000) divided by sampling steps.

13.5 Post-Processing

Multiple iterations will lead to redundant boxes requiring an added NMS to filter them.

14 Experiments and Results

14.1 Dataset

The experiments are conducted on the KITTI dataset [12], which is split into 3717 training and 3769 validation samples.

The average precision (AP) metric is used, where the IoU threshold is set to 0.7 for the car category.

All experiments are conducted on the car category with easy, moderate, and hard three levels.

14.2 Implementation Details

14.2.1 Voxelization Range

$$\text{X axis: } [0m, 70.4m] \quad (94)$$

$$\text{Y axis: } [-40m, 40m] \quad (95)$$

$$\text{Z axis: } [-3m, 1m] \quad (96)$$

14.2.2 Voxel Size

$$(0.05m, 0.05m, 0.1m) \quad (97)$$

14.2.3 Data Augmentation

Standard data augmentation techniques [42,19,36,37] are adopted, including:

- GT sampling
- Flipping
- Rotation
- Scaling

14.2.4 Training Configuration

- 2 NVIDIA RTX 3090 GPUs
- Batch size: 32
- Optimizer: AdamW [29]
- Learning rate policy: one-cycle

14.3 Main Results

One-step Diff3Det outperforms SECOND [42]:

- 1.42% on moderate level
- 6.97% on hard level

Diff3Det exceeds PointPillars [19]:

- 0.89% on moderate level
- 1.3% on hard level
- 8.08% on easy level

14.4 Multi-Step Sampling

When using multi-step sampling (step = 4), the performance improvement is mainly in the hard level of AP3D (0.37%).

The reason is that with the increase in sampling steps, the decoder generates more detection boxes, which is beneficial for detecting difficult samples.

15 Ablation Studies

15.1 Proposed Components

Table 2 shows the ablation studies on the diffusion-guided proposal generator.

15.1.1 Baseline

Diff3Det directly using proposals sampled from a Gaussian distribution for training and inference.

15.1.2 Adding Box Corruption

When adding boxes corrupted from the ground truth during training, there is a performance gain with mAP of 1.5% over the baseline.

15.1.3 Resampling Operation

The resampling procedure ensures each proposal box contains at least several points (e.g., 5 points). This brings an improvement of 0.99% on the easy level.

15.1.4 Size Correlation Strategy

The size correlation strategy to control the aspect of the size of 3D boxes brings a performance improvement of 0.71% mAP.

15.1.5 Dynamic Time Step

The dynamic time step makes the whole learning process easier, producing superior performance with mAP of 80.61% for all three levels.

15.2 Sampling Steps

Table 3 shows the effect of sampling steps in test.

Table 2: Effect of Sampling Steps (AP40)

Steps	Easy	Mod.	Hard	mAP
1	89.29	79.91	75.48	81.56
2	89.56	80.26	76.35	82.06
4	89.45	80.86	77.41	82.57
6	89.43	80.28	76.76	82.16
8	88.76	80.24	77.24	82.08

15.3 Hyperparameters

15.3.1 Signal Scale

Table 3: Ablation on Scale

Scale	Easy	Mod.	Hard	mAP
0.1	15.36	14.66	14.35	14.79
1.0	83.87	74.50	73.44	77.27
2.0	87.84	77.90	76.07	80.61

The signal scaling factor controls the signal-to-noise ratio (SNR) of the diffusion process. The value set to 2.0 yields the highest average precision (AP) performance.

15.3.2 Proposal Number

Table 4: Ablation on Proposal Number

N	Easy	Mod.	Hard	mAP
100	86.59	77.09	75.75	79.81
300	87.84	77.90	76.07	80.61
500	86.25	76.91	75.53	79.63

The performance achieves best when its value is set to 300.

15.3.3 Resampling Parameter η

Table 5: Ablation on η in Resample

η	Easy	Mod.	Hard	mAP
1	86.44	77.22	75.78	79.81
5	87.84	77.90	76.07	80.61
10	86.69	77.38	76.05	80.04

The parameter η controls the minimum number of point clouds for each proposal, which can effectively improve the quality of proposals. The best result is achieved when its value is set to 5.

16 Limitations and Conclusion

16.1 Limitations of Diff3Det

The primary limitation is that the proposed method poses difficulty for the decoder to regress prediction from random boxes, leading to a relatively slow convergence speed.

Besides, there remains scope for improving the performance of Diff3Det.

In the future, the authors would like to explore fast converging diffusion-based 3D object detection.

16.2 Conclusion

In this paper, a generative-based 3D object detection framework, Diff3Det, is proposed by viewing 3D object detection as a denoising diffusion process from noisy boxes to object boxes.

The key idea is to utilize the diffusion method to avoid the empirical heuristics setting of anchors.

The hope is that this method will provide new insights into the application of generative methods on 3D vision tasks.

17 Summary of Key Equations

Table 6: Key Equations in DiffusionDet and Diff3Det

Equation	Description
$q(z_t z_0) = \mathcal{N}(z_t; \sqrt{\bar{\alpha}_t}z_0, (1 - \bar{\alpha}_t)I)$	Forward diffusion
$\bar{\alpha}_t := \prod_{s=0}^t \alpha_s = \prod_{s=0}^t (1 - \beta_s)$	Noise schedule
$L_{\text{train}} = \frac{1}{2} \ f_\theta(z_t, t) - z_0\ ^2$	Training loss
$z_t = \sqrt{\bar{\alpha}_t}z_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$	Reparameterized diffusion
$q(z_{t-1} z_t, z_0) = \mathcal{N}(z_{t-1}; \tilde{\mu}(z_t, z_0), \tilde{\beta}_t I)$	Posterior distribution
$\tilde{\mu}_t(z_t, z_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} z_0 + \frac{\sqrt{\bar{\alpha}_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} z_t$	Posterior mean
$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$	Posterior variance
$p_\theta(z_{t-1} z_t) := \mathcal{N}(z_{t-1}; \mu_\theta(z_t, t), \Sigma_\theta(z_t, t))$	Learned reverse process
$W = \rho L + \sqrt{1 - \rho^2} X$	Correlation constraint (Diff3Det)
$C_{\text{match}} = \lambda_{\text{cls}} L_{\text{cls}} + \lambda_{\text{reg}} L_{\text{reg}} + \lambda_{\text{IoU}} L_{\text{BEV IoU}}$	Matching cost
C	Optimal assignment
$\arg \min_{i \in M, j \in N} C_{\text{match}}(\hat{y}_i, y_j)$	
$L = \lambda_{\text{cls}} L_{\text{cls}} + \lambda_{\text{reg}} L_{\text{reg}} + \lambda_{\text{IoU}} L_{\text{DIOU}}$	Training loss
x_{t-s}	DDIM sampling
$\sqrt{\alpha_{t-s}} \left(\frac{x_t - \sqrt{1 - \alpha_t} \epsilon_\theta^{(t)}(x_t)}{\sqrt{\alpha_t}} \right) + \sqrt{1 - \alpha_{t-s} - \sigma_t^2} \epsilon_\theta^{(t)}(x_t) + \sigma_t \epsilon_t$	
$\sigma_t = \sqrt{\frac{1 - \alpha_t / \alpha_{t-s}}{(1 - \alpha_{t-s}) / (1 - \alpha_t)}}$	Noise schedule in sampling

18 The Big Picture

18.1 DiffusionDet Training

$$\boxed{\text{GT Boxes } z_0 \rightarrow \text{Add Noise} \rightarrow \text{Noisy Boxes } z_t \rightarrow \text{Decoder} \rightarrow \text{Predict Clean Box } \hat{z}_0 \rightarrow \text{Compare with } z_0} \quad (98)$$

18.2 DiffusionDet Inference

$$\boxed{\text{Random Boxes } z_T \rightarrow \text{Decoder} \rightarrow \text{Refined Boxes} \rightarrow \text{DDIM} \rightarrow \text{Next Boxes} \rightarrow \cdots \rightarrow \text{Final Boxes}} \quad (99)$$

18.3 Diff3Det Training

$$\boxed{\text{GT Boxes } x_0 \rightarrow \text{Add Noise} \rightarrow \text{Noisy Boxes } x_t \rightarrow \text{Decoder} \rightarrow \text{Predict Clean Box } \hat{x}_0 \rightarrow \text{Compare with } x_0} \quad (100)$$

18.4 Diff3Det Inference

$$\boxed{\text{Random Boxes } x_T \rightarrow \text{Decoder} \rightarrow \text{Refined Boxes} \rightarrow \text{DDIM} \rightarrow \text{Next Boxes} \rightarrow \cdots \rightarrow \text{Final Boxes}} \quad (101)$$

19 Additional Experimental Details and Results

19.1 DiffusionDet: Additional Experiments

19.1.1 Dynamic Number of Boxes: Training vs Inference Matching

Table 5 shows the matching between training and inference box numbers on COCO. DiffusionDet decouples the number of boxes during the training and inference stages and works well with flexible combinations.

Table 7: Matching between training and inference box numbers on COCO.

Train	Eval 100	Eval 300	Eval 500	Eval 1000	Eval 2000
100	42.9	44.4	44.5	44.6	44.6
300	42.8	45.7	46.2	46.3	46.4
500	41.9	45.8	46.3	46.7	46.8

First, no matter how many random boxes DiffusionDet uses for training, the accuracy increases steadily with the N_{eval} until the saturated point at around 2000 random boxes.

Second, DiffusionDet tends to perform better when the N_{train} and N_{eval} matches with each other. For example, DiffusionDet trained with $N_{\text{train}} = 100$ boxes behaves better than $N_{\text{train}} = 300$ and 500 when $N_{\text{eval}} = 100$.

19.1.2 Running Time vs. Accuracy

Table 6 investigates the running time of DiffusionDet under multiple settings, evaluated on a single NVIDIA A100 GPU with a mini-batch size of 1. The notation $\#Stages \times \#Heads$ indicates the number of stages and heads utilized during the training and test phases, as depicted in Figure 2b.

Table 8: Running time vs. performance comparison.

Method	Train Test	COCO	CrowdHuman	FPS
DETR [10]	$6 \times 1 \rightarrow 6 \times 1$	42.0	61.3	39
	$6 \times 1 \rightarrow 6 \times 2$	41.6 (-0.4)	62.5 (+1.2)	32
Sparse R-CNN [91]	$6 \times 1 \rightarrow 6 \times 1$	45.0	66.6	30
	$6 \times 1 \rightarrow 6 \times 2$	43.6 (-1.4)	60.6 (-6.0)	21
	$12 \times 1 \rightarrow 12 \times 1$	44.7 (-0.3)	66.1 (-0.5)	21
DiffusionDet	$6 \times 1 \rightarrow 6 \times 1$	45.8	66.6	30
	$6 \times 1 \rightarrow 6 \times 2$	46.5 (+0.7)	69.7 (+3.1)	20
DiffusionDet †	$6 \times 1 \rightarrow 6 \times 1$	46.8 (+1.0)	71.0 (+4.4)	24

† denotes DiffusionDet with 1000 boxes.

DiffusionDet with a single iteration step and 300 evaluation boxes demonstrates a comparable speed to Sparse R-CNN, achieving 30 and 31 frames per second (FPS), respectively.

When increasing the number of iteration steps, DiffusionDet achieves a 0.7 AP gain on COCO and a 3.1 AP gain on CrowdHuman. DiffusionDet obtains clear performance gains with 1000 evaluation boxes.

However, neither DETR nor Sparse R-CNN can achieve performance gains with additional iteration steps. Even if we expand the number of stages to 12, it can cause performance degradation for Sparse R-CNN.

19.1.3 Full Tuning on CrowdHuman

Table 7 shows the full-tuning results on CrowdHuman.

Table 9: Full tuning on CrowdHuman.

Method	AP50 ↑	mMR ↓	Recall ↑
Faster R-CNN [74]	85.0	50.4	90.2
RetinaNet [56]	81.7	57.6	88.6
FCOS [94]	86.1	55.2	94.3
DETR [10]	66.1	80.6	-
Deformable DETR [115]	86.7	54.0	92.5
Sparse R-CNN [91] (500)	89.2	48.3	95.9
Sparse R-CNN [91] (1000)	89.7	49.1	97.5
DiffusionDet (1 @ 1000)	90.1	46.5	96.2
DiffusionDet (1 @ 3000)	91.2	47.7	98.4
DiffusionDet (3 @ 1000)	91.4	45.7	98.4

DiffusionDet achieves superior performance compared with previous methods. With a single step and 1000 boxes, DiffusionDet obtains 90.1 AP50, outperforming Sparse R-CNN with 1000 boxes.

Further increasing boxes to 3000 and iteration steps can both bring performance gains.

19.1.4 Random Seed Stability

Since DiffusionDet is given random boxes as input at the start of inference, one may ask whether there is a large performance variance across different random seeds.

The paper evaluates the stability of DiffusionDet by training five models independently with the same configurations except for random seed. Then, each model instance is evaluated with ten different random seeds.

As shown in Figure 4, most evaluation results are distributed closely to 45.7 AP. Besides, the performance differences among different model instances are marginal, demonstrating that DiffusionDet is robust to the random boxes and produces reliable results.

19.1.5 Sampling Strategy: DDIM and Box Renewal

The paper compares different sampling strategies in Table 4c.

Table 10: Comparison of sampling strategies.

Strategy	Step 1	Step 3	Step 5
No DDIM, No Renewal	45.8	44.2	42.9
Only DDIM	45.8	45.1	44.6
Only Renewal	45.8	45.3	44.9
DDIM + Renewal	45.8	46.2	46.5

When evaluating DiffusionDet that does not use DDIM, directly taking the output prediction of the current step as input for the next step degrades performance with more iteration steps.

Only using DDIM or box renewal brings slight benefits at 3 iteration steps.

DiffusionDet attains remarkable gains when equipped with both DDIM and renewal. These experiments together verify the necessity of both DDIM and box renewal in the sampling step.

19.2 Diff3Det: Additional Experimental Details

19.2.1 Qualitative Results

Figure 5 shows qualitative results of Diff3Det on the KITTI validation set. The prediction boxes are shown in red and ground truth boxes are shown in green. Diff3Det demonstrates accurate localization and orientation estimation for objects in the point cloud.

19.2.2 AP40 Metrics

The paper reports AP40 (average precision calculated by 40 recall positions) in addition to the standard AP11 (11 recall positions).

The 11 recall positions are:

$$0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0 \quad (102)$$

At each recall level, the maximum precision obtained for any recall $\geq$ that level is taken:

$$P_{\text{interp}}(r) = \max_{\tilde{r} \geq r} P(\tilde{r}) \quad (103)$$

The AP40 metric uses 40 recall positions which provides a more accurate evaluation. The paper notes that recall 40 metrics are more accurate [38] and provide a better reflection of the effectiveness of the iterative approach.

19.2.3 Comparison with Multi-Modal Methods

Table 1 in the paper compares Diff3Det with both LiDAR-only and multi-modal methods:

- **LiDAR-only methods:** PointPillars [19], VoxelNet [49], SECOND [42], TAnet [28]
- **Multi-modal methods (RGB + LiDAR):** MV3D [8], ContFuse [23], AVOD-FPN [18], F-PointNet [31]

Diff3Det achieves competitive performance compared with both categories, demonstrating the effectiveness of the diffusion-based approach even without using RGB images.

19.2.4 Dynamic Head Extension to 3D

Diff3Det utilizes the dynamic head [40], extended to 3D object detection. The approach does not rely on the learnable query and learnable embedding for dynamic convolution prediction but instead adopts randomly selected proposal boxes, temporal noise levels, and RoI features.

19.2.5 GT Sampling Augmentation

The paper adopts standard data augmentation techniques [42,19,36,37], including:

- GT sampling (copying ground truth objects into the scene)
- Flipping
- Rotation
- Scaling

19.2.6 AP11 vs AP40 Difference

The paper notes that some metrics decreased with AP11 (calculated by 11 recall positions) in Table 1, while AP40 showed improvements. The authors believe it is due to the fact that recall 40 metrics are more accurate [38] and provide a better reflection of the effectiveness of the iterative approach.

The slight performance drop in the recall 11 metrics should not lead to any misunderstanding.

19.2.7 Detailed Component Ablation for Diff3Det

Table 2 shows the ablation studies on the diffusion-guided proposal generator:

Table 11: Ablation of proposed components in Diff3Det.

Configuration	Easy	Mod.	Hard	mAP
Baseline (Gaussian proposals)	86.50	76.20	74.80	79.17
+ GT corruption	87.10	77.60	75.90	80.20
+ Resampling	88.09	77.80	76.20	80.70
+ Size correlation	88.50	78.10	76.50	81.03
+ Dynamic time step	87.84	77.90	76.07	80.61

The baseline is Diff3Det directly using proposals sampled from a Gaussian distribution for training and inference.

When adding boxes corrupted from the ground truth during training, there is a performance gain with mAP of 1.5% over the baseline.

The resampling procedure to ensure each proposal box contains at least several points (e.g., 5 points) brings an improvement of 0.99% on the easy level.

The size correlation strategy to control the aspect of the size of 3D boxes brings a performance improvement of 0.71% mAP.

The dynamic time step makes the whole learning process easier, producing superior performance with mAP of 80.61% for all three levels.

19.2.8 Multi-Step Sampling Analysis

The paper analyzes the effect of multi-step sampling:

Table 12: Effect of sampling steps in test (AP40).

Steps	Easy	Moderate	Hard	mAP
1	89.29	79.91	75.48	81.56
2	89.56	80.26	76.35	82.06
4	89.45	80.86	77.41	82.57
6	89.43	80.28	76.76	82.16
8	88.76	80.24	77.24	82.08

The average precision (AP) calculated by 40 recall positions exhibits varying degrees of improvement. Notably, the performance at the moderate level is enhanced by 0.95%, while the hard level shows an improvement of 1.93% when comparing sampling step = 4 to step = 1.

19.3 Summary of Key Experimental Findings

Table 13: Summary of key findings from DiffusionDet and Diff3Det experiments.

Finding	Description
Dynamic boxes	DiffusionDet allows $N_{\text{train}} \neq N_{\text{eval}}$ without retraining
Iterative refinement	More refinement steps improve performance with ensemble
Zero-shot transfer	DiffusionDet transfers from COCO to CrowdHuman effectively
Signal scale	Scale = 2.0 works best for box diffusion
Proposal number	$N = 300$ gives best trade-off for Diff3Det
Resampling	$\eta = 5$ points per proposal works best
Correlation coefficient	$\rho = 0.8$ improves proposal quality
Dynamic time step	Sine schedule improves training stability
AP40 vs AP11	AP40 provides more accurate evaluation
Multi-modal methods	Diff3Det competitive with RGB+LiDAR methods